

AURALIS: A Self-Mapping Smart Cane with Spatio-Temporal Hazard Memory and Neural Sensor Fusion for Visually Impaired Pedestrians

Abstract—Electronic travel aids for visually impaired people are mostly reactive: they warn about an obstacle only while it is in front of the sensor and forget it immediately, although most daily walks repeat the same routes and meet the same hazards. We present AURALIS, a low-cost smart cane that *remembers*. A Raspberry Pi 4 on the cane runs a YOLOv8 convolutional detector on a USB camera and an HC-SR04 ultrasonic ranger, estimates distance from a single camera, and decides what to announce. The user’s smartphone adds GPS and compass, speaks the alerts in English, Hindi, Telugu or Tamil through Bluetooth headphones, and maintains a spatio-temporal hazard memory: every hazard is stored at its estimated position, its risk grows with repeated sightings and decays with a seven-day half-life, and remembered hazards are announced up to 25 m ahead, before they are visible. The phone also builds a pedestrian graph from the user’s own walks, needing no map data, and plans routes with a risk-weighted A* search. A 484-parameter neural network fuses vision, ultrasonic and memory features into four alert levels with a hard safety override for drops. In this draft we report the implemented system and a preliminary evaluation: the fusion network reaches 0.890 accuracy and 0.933 urgent recall on a held-out synthetic set (rule baseline: 0.768 and 0.865), risk-aware routing cuts route risk by 89% for a $2.2\times$ longer path on a campus graph, and memory raises the warning lead time from about 2.5 s to about 20 s by design. Field experiments are planned and outlined.

Index Terms—assistive technology, smart cane, visually impaired navigation, object detection, YOLO, sensor fusion, spatio-temporal memory, risk-aware routing, edge AI

I. INTRODUCTION

About 2.2 billion people live with a vision impairment, and at least 43 million are blind [1], [2]. The white cane remains the most common mobility aid: it is cheap and reliable, but it only reaches about one metre ahead, at ground level. Electronic travel aids add ultrasonic or camera sensing [3], [4], [6], and recent smart canes run deep object detectors such as YOLO [7] on embedded boards.

These systems share one limitation: they are *reactive*. A pothole on the way to the library is detected only when it is a few metres away, every single day, and is forgotten as soon as it leaves the camera view. Yet a pedestrian’s daily walks are highly repetitive, and the most dangerous hazards in Indian streets and campuses—open drains, potholes, broken steps, low branches—are static for days or weeks.

AURALIS turns the cane from a reactive sensor into a system that *learns its user’s environment*. Our contributions are:

- 1) **Spatio-temporal hazard memory** that combines repeated evidence with temporal decay and user correction, and warns about remembered hazards long before they come into view (Sec. IV).
- 2) **Infrastructure-free self-mapping**: a pedestrian graph built from the user’s own GPS breadcrumbs, with **risk-aware A*** routing on it (Sec. V).
- 3) **Neural late fusion** of camera, ultrasonic and memory features into a four-level alert, with a hard safety override (Sec. VI).
- 4) A **minimal, low-cost implementation**: one Raspberry Pi 4, one USB camera and one ultrasonic sensor on the cane; GPS, voice in four Indian-context languages and haptics on the user’s phone (Sec. III).

II. RELATED WORK

Sensor canes. Surveys of electronic travel aids [3]–[6] cover ultrasonic, infrared, laser and camera-based canes and wearables. Ultrasonic canes are cheap and robust but cannot say *what* an obstacle is; camera systems recognise objects but are sensitive to light and need distance estimation.

Vision-based aids. Single-stage detectors [7], [8] trained on COCO [9] make real-time recognition feasible on embedded CPUs, and monocular depth networks such as MiDaS [10] give relative depth from one camera. Most vision aids announce what is currently visible.

Maps and routing. Crowdsourced accessibility mapping such as Project Sidewalk [11] and OpenStreetMap [12] record sidewalk problems, but depend on volunteers and are sparse outside large cities. Classic A* [13] is widely used for path planning.

Gap. We are not aware of a low-cost cane that (i) remembers detected hazards at their location with evidence accumulation and decay, (ii) builds its own walkable graph without external maps, and (iii) uses that memory both for early warning and for risk-aware routing. [Add 5–8 recent (2021–2025) smart-cane papers from IEEE Xplore and compare in a table.]

III. SYSTEM OVERVIEW

Fig. 1 shows the architecture. The **cane** carries a Raspberry Pi 4 (4 GB), a USB webcam (60° horizontal field of view, 640×480) about 0.9 m above the ground, an HC-SR04 ultrasonic ranger and a 5 V/3 A power bank. The **phone** joins the cane over the phone’s own Wi-Fi hotspot and opens the cane’s web application over HTTPS (needed for GPS in

TABLE I
OBSTACLE TAXONOMY (FROM `PI/CONFIG.PY`)

Category	Example classes	Remembered
Drop	pothole, open drain, stairs down, curb	yes; STOP <1.5 m
Raised	stairs up, speed breaker	yes
Static	pole, tree, wall, bench, chair, parked vehicle	yes
Head height	branch, signboard, high and close objects	yes
Moving	person, bicycle, car, auto-rickshaw, dog, cow	no
Zone	construction	yes

mobile browsers); obstacles stream on a secure WebSocket, and the phone streams its GPS fix back once per second. The cane’s memory is stored on the Pi, so a second screen (e.g. a projector or a caregiver’s laptop) can follow the user’s live position.

A. Obstacle taxonomy and walking corridor

Detections are mapped to six categories (Table I) that decide severity, the wording of the alert, and whether the object is remembered. Moving objects are announced but never stored. A vehicle whose estimated distance stays constant for 3 s is re-classified as a parked, static obstacle. The image is split into left, ahead and right thirds; an object is *in the walking corridor* when its box overlaps the middle third and it is within 4 m.

B. Detection (NN 1)

The detector is YOLOv8n [8] exported to ONNX and executed with ONNX Runtime [15] on the Pi’s four CPU cores, at 480 px (or 320 px in a fast mode). The stock COCO model [9] already covers people, vehicles, animals and street furniture. Classes specific to Indian streets (pothole, open drain, stairs, curb, speed breaker) are trained with a Colab pipeline that merges public datasets with photos collected from the cane; the cane loads the custom model automatically when present. [\[Report the custom model: dataset size per class, mAP@0.5, precision, recall.\]](#)

An object is announced only when it is stable in at least two of the last three frames; it is repeated after 5 s or when it comes more than 1 m closer, and an object closer than 0.8 m, or any drop, interrupts current speech.

C. Distance from one camera

Two geometric estimates are available. *Ground-plane geometry* uses the image row y_b of the box’s bottom edge: with camera height h , downward tilt θ and focal length f (in pixels),

$$d = \frac{h}{\tan(\theta + \arctan \frac{y_b - H/2}{f})}. \quad (1)$$

Known size uses a typical real height H_c per class: $d = H_c f / h_{\text{box}}$. A monocular depth network [10] can fill in objects neither method handles, its relative depth being re-scaled to metres every frame by fitting it to the geometric estimates.

TABLE II
WARNING DISTANCE AND LEAD TIME (BY DESIGN, 1.2 M/S)

Source of the warning	Distance	Lead time
Camera alone (announce range)	≤ 3 m	≈ 2.5 s
Hazard memory (look-ahead)	≤ 25 m	≈ 20.8 s
Measured in the field (E4)	[m]	[s]

Fig. 2 shows why we default to the size estimate: with our mounting ($h = 0.9$ m, $\theta = 20^\circ$, $f = 554$ px) a tilt error of only $\pm 2^\circ$ —easily caused by holding the cane differently—moves the ground-plane estimate by $-6.8/ + 7.3\%$ at 1 m and $-11.4/ + 14.4\%$ at 3 m, and the ground is not visible closer than 0.95 m. The size estimate is independent of how the cane is held. Its error equals the error in box height plus the spread of real object heights, which we will measure (E1). For whatever is straight ahead, the forward ultrasonic ranger replaces the camera’s estimate when they agree within 1 m, and on its own announces any unrecognised obstacle within 1 m.

IV. SPATIO-TEMPORAL HAZARD MEMORY

When an alert of level *warning* or higher concerns a rememberable object and the GPS fix is better than 30 m, the hazard is stored at the user’s position projected d metres along the compass heading. Sightings of the same class within 8 m are merged (position averaged with weight 3:1, at most one new sighting per minute so that standing still does not inflate the count). The risk of hazard i is

$$r_i(t) = s_i \cdot (1 - e^{-n_i/2}) \cdot 0.5^{\Delta t_i/T} \cdot c_i, \quad (2)$$

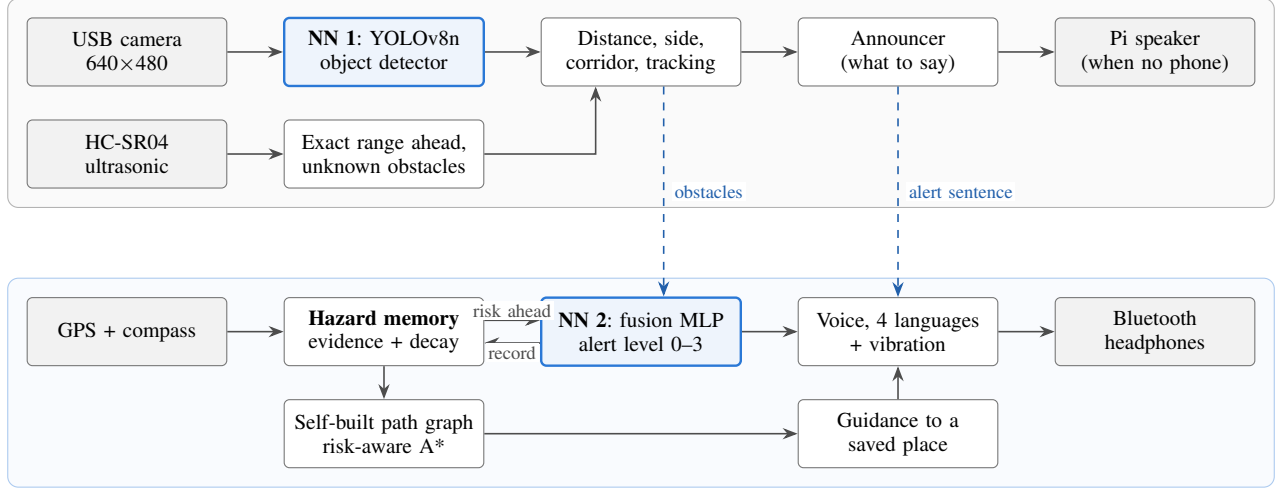
where s_i is the category severity, n_i the number of sightings, Δt_i the time since the last sighting, $T = 7$ days the half-life and c_i the detection confidence. One sighting gives at most 39% of full risk, three give 78%, five 92%. A hazard that is no longer seen fades below the warning threshold of 0.15 after 9.7, 16.6 and 18.3 days for 1, 3 and 5 sightings (Fig. 3). The user can also say a hazard is gone, which deletes all hazards within 10 m.

Early warning. While walking, the phone looks 25 m ahead within $\pm 45^\circ$ of the heading and announces a remembered hazard once per two minutes, e.g. “*Careful. Remembered pothole, 20 metres ahead*”. At a walking speed of 1.2 m/s this gives about 20 s of warning, compared with about 2.5 s from the camera, which announces corridor obstacles within 3 m (Table II).

V. SELF-BUILT MAP AND RISK-AWARE ROUTING

The phone records a breadcrumb every 5 m walked, ignoring fixes worse than 25 m. Breadcrumbs within 6 m become one graph node, and consecutive nodes are joined by an edge unless they are more than 30 m apart (a GPS jump). The result is a graph of paths the user has actually walked, with no external map. The user can name places (“Library”, “Canteen”) by voice and ask to be guided there. Each edge e

Cane: Raspberry Pi 4



Phone: web app over HTTPS, GPS, voice

Fig. 1. AURALIS architecture. The cane decides *what* is ahead and what to say; the phone adds location, memory, routing and the user-facing voice. Dashed arrows travel over a secure WebSocket on the phone's Wi-Fi hotspot; the phone also sends its GPS position back to the cane. Neural components are outlined in blue.

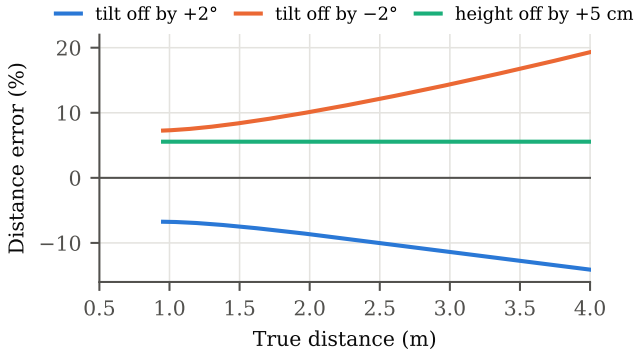


Fig. 2. Error of the ground-plane distance estimate for small mounting errors (analytic, from the deployed camera parameters). The size-based estimate and the ultrasonic range do not depend on tilt.

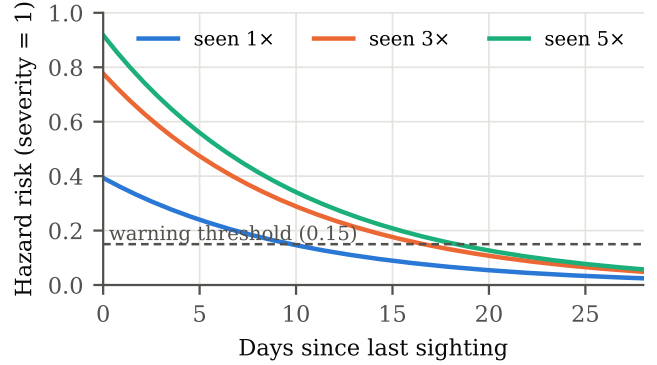


Fig. 3. Hazard risk over time without new sightings, for a hazard seen 1, 3 or 5 times (Eq. 2, severity and confidence 1). Repeated evidence keeps a hazard active for longer.

of length ℓ_e gets the risk ρ_e of the remembered hazards within 10 m of it, and A* [13] minimises

$$\text{cost}(e) = \ell_e (1 + \lambda \rho_e). \quad (3)$$

On an example campus block with five remembered hazards (Fig. 4), the shortest route ($\lambda \leq 2$) is 90.8 m long and crosses an open drain and a pothole (length-weighted risk 0.458). From $\lambda = 4$ the planner switches to a 202.7 m detour with risk 0.051: **89% less risk for 2.2× the distance**. The app shows both routes so the user can choose; $\lambda = 4$ is the default.

VI. NEURAL SENSOR FUSION

The alert level (safe, caution, warning, urgent) is decided by a multilayer perceptron with layers $8 \rightarrow 16 \rightarrow 16 \rightarrow 4$ (ReLU, softmax; 484 parameters) that runs in the phone's browser in about $0.5 \mu\text{s}$ per frame. Its eight inputs are: the

nearest corridor obstacle's distance, the ultrasonic drop signal, the object's severity, detection confidence, box area and lateral offset, the summed risk of remembered hazards within 15 m ahead, and the approach speed from tracking. Two safety rules sit outside the network: a drop closer than 1.5 m is always *urgent* (the network can raise but never lower this), and if the cane's data stops for more than 3 s the phone says so and vibrates.

Training. Labelled field data is expensive, so the network is first trained on 20,000 synthetic situations labelled by an expert rule set that combines distance, threat (severity $\times$ confidence, reduced off-path), apparent size, approach speed and memory risk, with 3% label noise; Adam [14], 60 epochs. A *research mode* in the app lets a sighted helper label real situations while walking; these rows are weighted 5× when re-training.

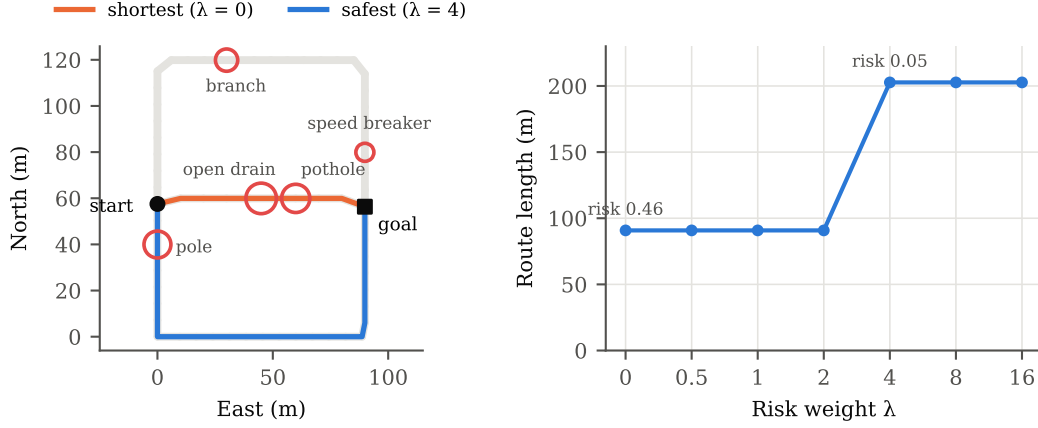


Fig. 4. Risk-aware routing with the app’s own code on the example campus graph. Left: shortest and safest route; circle size is hazard risk. Right: route length versus the risk weight λ , annotated with the route’s length-weighted risk.

TABLE III
FUSION NETWORK VS. RULE BASELINE (HELD-OUT SYNTHETIC SET, $n = 10,000$)

Model	Acc.	Macro F1	Urgent rec.	Urgent prec.
Distance rules	0.768	0.754	0.865	0.988
Fusion MLP	0.890	0.890	0.933	0.917
Field data (E2)	□	□	□	□

True level	Safe	0.92	0.07	0.00	0.00
	Caution	0.05	0.88	0.06	0.00
	Warning	0.01	0.12	0.82	0.05
	Urgent	0.00	0.00	0.06	0.93
		Safe	Caution	Warning	Urgent

Predicted level

Fig. 5. Row-normalised confusion matrix of the fusion network on the held-out synthetic set. Only 7 of 1,194 urgent situations (0.6%) are predicted as safe or caution; most errors are between neighbouring levels.

Preliminary result. On 10,000 fresh synthetic situations (Table III, Fig. 5) the network reaches 0.890 accuracy and 0.933 urgent recall. A distance-only rule baseline—the app’s fallback when no network is loaded—reaches 0.768 and 0.865, and misses most *warning* situations (recall 0.365 vs. 0.824), because they depend on what the object is and how it moves, not only on how far it is. We stress that these labels come from the rule teacher, so this measures how well the compact network captures a richer decision rule; field accuracy is experiment E2.

VII. IMPLEMENTATION

The cane software is Python (OpenCV, ONNX Runtime, aiohttp) and starts with one command; it serves the phone application, the annotated camera stream for a helper and the memory store. The phone application is a responsive web app (Fig. 6) with four screens—Walk, Places, Memory, Settings—fully translated into English, Hindi, Telugu and Tamil. Speech uses neural text-to-speech voices on the Pi, cached for offline use, or the phone’s own voices; when a phone is connected the alerts are spoken on the phone, so that they reach the user’s Bluetooth headphones, and the Pi speaker falls silent. Holding a hand in front of the ultrasonic sensor for 2 s pauses or resumes the cane without touching the phone. The map uses Leaflet with OpenStreetMap tiles [12]; paths, hazards and places are still drawn when offline.

On a development laptop CPU the detector takes 10.5 ms (320 px) and 22.4 ms (480 px) per frame. [Measure on the Pi 4: detector latency, end-to-end frames per second, and alert latency (E3).]

VIII. PLANNED EVALUATION

Table IV lists the field experiments that complete this study. All of them can be run on campus with the existing hardware and the app’s research mode and log export.

IX. LIMITATIONS AND FUTURE WORK

GPS accuracy (typically 5–10 m outdoors, worse near buildings) limits where a remembered hazard can be placed; merging repeated sightings and the 8 m merge radius partly compensate. Indoors there is no GPS, and memory only works outdoors. The fusion network is so far trained on synthetic data, and the stock detector does not yet know potholes, drains and stairs; both are addressed by the planned field data. Memory is per user; sharing anonymised hazards between users of the same campus would let one user’s sighting protect others, and is our main next step, together with a native app that keeps working with the screen off.

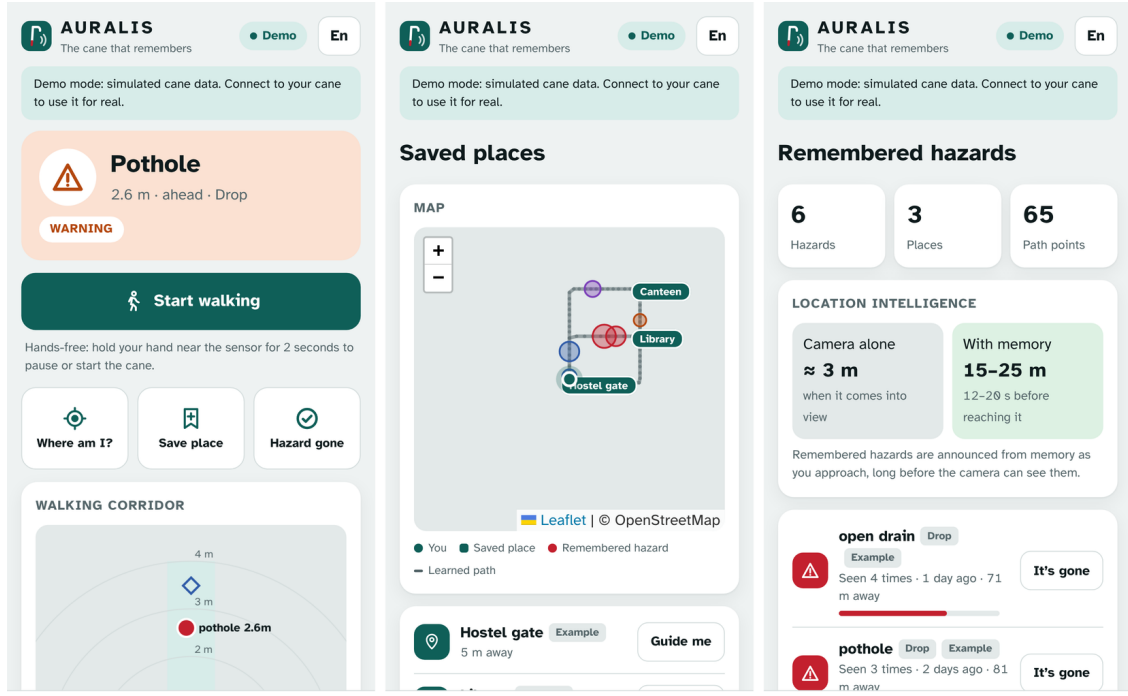


Fig. 6. Phone application (demo mode with simulated sensor data, map background not loaded). Left: Walk screen with the current alert and the top-down walking corridor; the hollow diamond is a remembered hazard ahead. Middle: self-built path graph with remembered hazards and saved places. Right: hazard memory with evidence counts, age and risk.

TABLE IV
FIELD EXPERIMENTS (TO BE COMPLETED)

#	Experiment	Metric
E1	Distance vs. tape measure at 0.5/1/2/3 m, 10 readings each, size vs. ground vs. ultrasonic	MAE, std
E1c	Custom detector on held-out campus photos	P, R, mAP@0.5
E2	Fusion network vs. rules on labelled walks (80/20)	accuracy, urgent recall
E3	Obstacle appears → voice starts (video)	latency (ms)
E4	Walk one route 5× past a fixed hazard	warning distance, lead time
E5	Shortest vs. safest on two real campus routes	length, risk
E7	5 listeners rate each language	intelligibility (1–5)

X. CONCLUSION

AURALIS shows that a commodity smart cane can do more than react: by remembering where hazards are, learning the user’s own paths and fusing what it sees with what it remembers, it can warn several times earlier and route around known dangers, while keeping the hardware to one board, one camera and one ultrasonic sensor, and the user’s own phone and headphones.

ACKNOWLEDGMENT

[Guide, department and lab.]

REFERENCES

- [1] World Health Organization, *World Report on Vision*. Geneva, Switzerland: WHO, 2019.
- [2] R. R. A. Bourne *et al.*, “Magnitude, temporal trends, and projections of the global prevalence of blindness and distance and near vision impairment: a systematic review and meta-analysis,” *Lancet Global Health*, vol. 5, no. 9, pp. e888–e897, 2017.
- [3] D. Dakopoulos and N. G. Bourbakis, “Wearable obstacle avoidance electronic travel aids for blind: a survey,” *IEEE Trans. Syst., Man, Cybern. C*, vol. 40, no. 1, pp. 25–35, 2010.
- [4] W. Elmannai and K. Elleithy, “Sensor-based assistive devices for visually-impaired people: current status, challenges, and future directions,” *Sensors*, vol. 17, no. 3, p. 565, 2017.
- [5] S. Real and A. Araujo, “Navigation systems for the blind and visually impaired: past work, challenges, and open problems,” *Sensors*, vol. 19, no. 15, p. 3404, 2019.
- [6] B. Kuriakose, R. Shrestha, and F. E. Sandnes, “Tools and technologies for blind and visually impaired navigation support: a review,” *IET Tech. Rev.*, vol. 39, no. 1, pp. 3–18, 2022.
- [7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: unified, real-time object detection,” in *Proc. IEEE CVPR*, 2016, pp. 779–788.
- [8] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [9] T.-Y. Lin *et al.*, “Microsoft COCO: common objects in context,” in *Proc. ECCV*, 2014, pp. 740–755.
- [10] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, “Towards robust monocular depth estimation: mixing datasets for zero-shot cross-dataset transfer,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, 2022.
- [11] M. Saha *et al.*, “Project Sidewalk: a web-based crowdsourcing tool for collecting sidewalk accessibility data at scale,” in *Proc. ACM CHI*, 2019.
- [12] M. Haklay and P. Weber, “OpenStreetMap: user-generated street maps,” *IEEE Pervasive Comput.*, vol. 7, no. 4, pp. 12–18, 2008.
- [13] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

- [14] D. P. Kingma and J. Ba, "Adam: a method for stochastic optimization," in *Proc. ICLR*, 2015.
- [15] ONNX Runtime developers, "ONNX Runtime," 2021. [Online]. Available: <https://onnxruntime.ai>